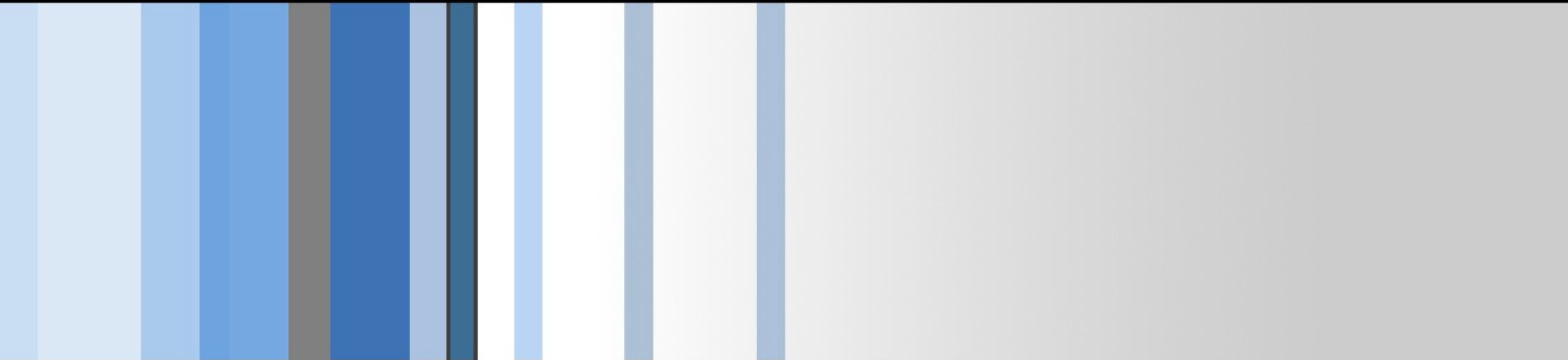
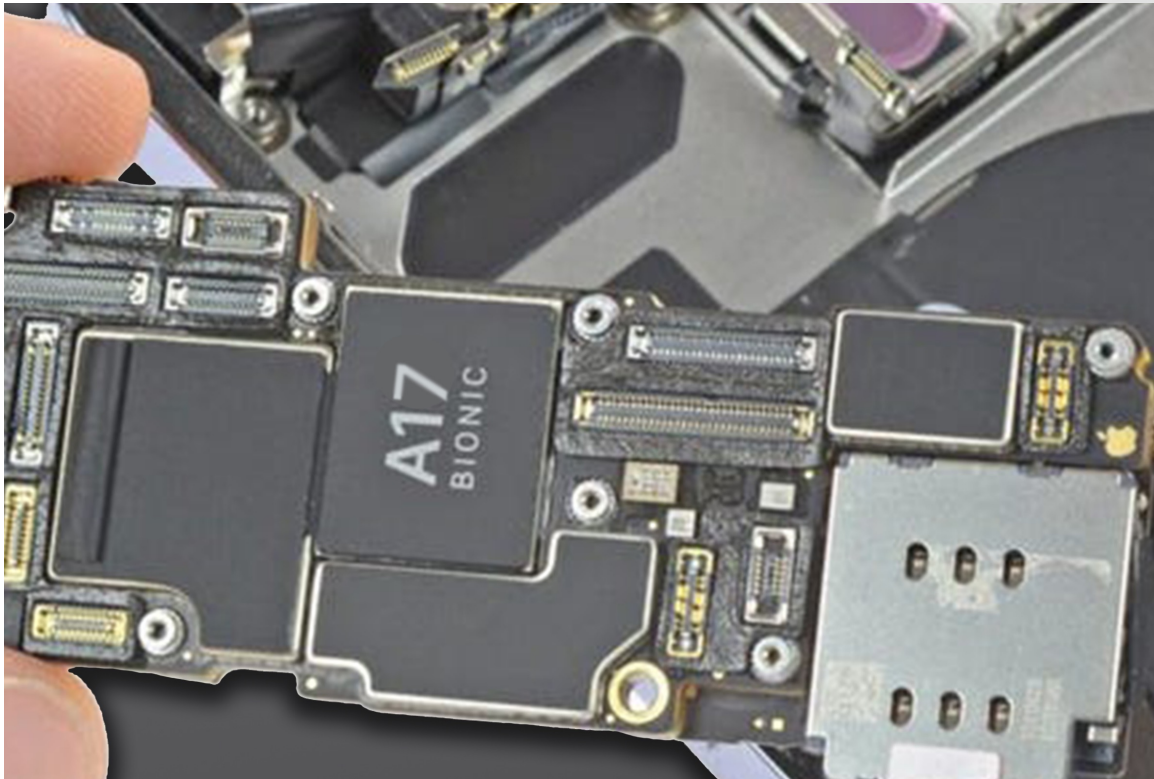


CSC 496: iOS App Development
Concurrency and Threading, Asynchronous Programming
Si Chen (schen@wcupa.edu)



Introduction

- What's Concurrency?
 - Doing multiple tasks at the same time



Apple A17 Pro

General information

Launched	September 12, 2023
Designed by	Apple Inc.
Common manufacturer(s)	TSMC ^[1]
Product code	APL1V02
Max. CPU clock rate	to 3.78 GHz ^[2]

Cache

L2 cache	16 MB (performance cores) 4 MB (efficiency cores)
L4 cache	24 MB (system level cache)

Architecture and classification

Application	Mobile
Technology node	3 nm (N3B)
Instruction set	AArch64

Physical specifications

Transistors	19 billion
Cores	6 (4 efficiency, 2 performance) ^[3]
GPU(s)	Apple-designed 6-core GPU ^[3]

History

Predecessor(s)	Apple A16
-----------------------	---------------------------

A-series chips

- Apple's A-series chips are custom-designed chips for the iPhone, iPad, and other Apple devices. Starting from certain versions, these chips contain two different types of cores: **efficiency cores (E-cores)** and **performance cores (P-cores)**.

1. Efficiency (E) cores:

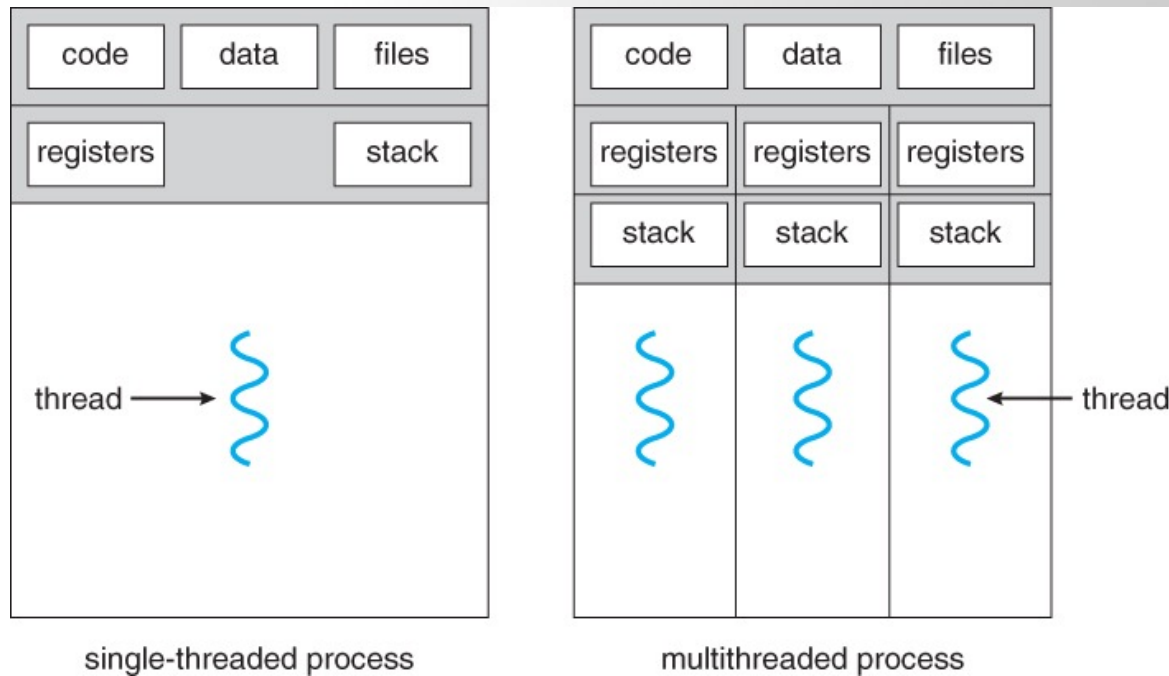
1. **Design Objective:** These are designed for energy conservation, capable of running consistently even under high loads.
2. **Usage:** For handling background tasks, tasks with low computational demands, or tasks that run continuously.
3. **Characteristics:** They have relatively lower processing power, but they're more energy-efficient.

2. Performance (P) cores:

1. **Design Objective:** Designed for handling high-load tasks, such as complex graphics rendering or computation-intensive tasks.
2. **Usage:** For tasks that demand a high computational capability within a short time.
3. **Characteristics:** They process tasks faster but also consume more power.

Threads

- Every iOS application has a **main thread**, which is there to **display your user interface and listen for events**
- Complex computations may slow down the main thread and freeze the app.
 - Here is where multithreading comes into play.
 - move all heavy lifting to a **background thread** and then move the result back to the main once done



Introduction



main thread and background thread

Introduction



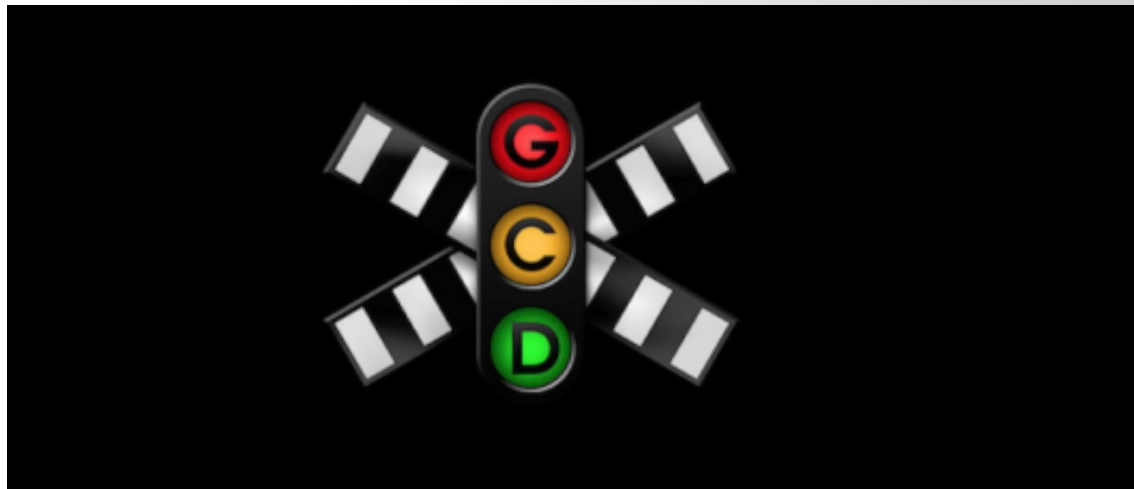
Advanced concurrency models



- In iOS, **threads** can be used to achieve concurrency and multitasking. However, Apple recommends using more advanced concurrency models, such as **Grand Central Dispatch (GCD)** and **Operation Queues**, because they are easier to manage and have a smaller overhead.

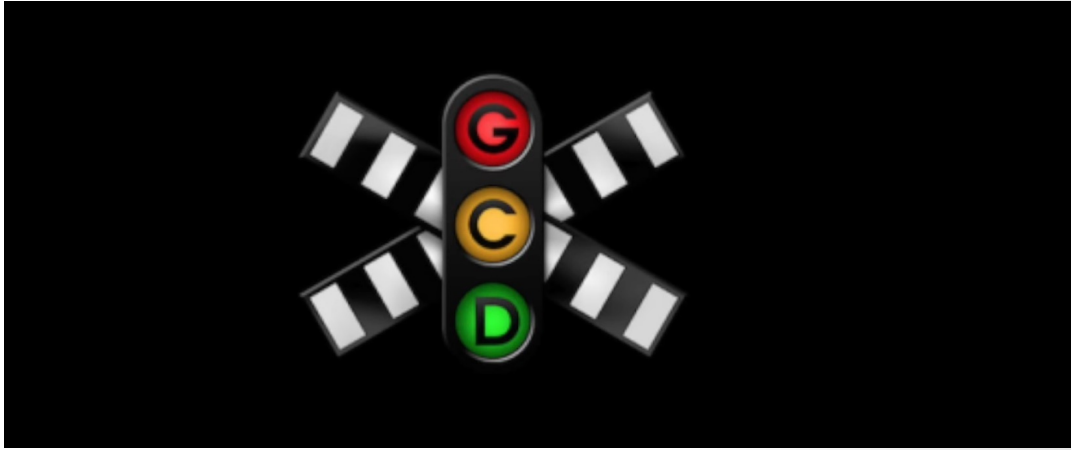
Queues

- A queue is just a bunch of code blocks, lined or queued up, waiting for a thread to execute them.
- You don't need to worry about threads in Swift, only queues.
 - The iOS system takes care of providing/allocating threads to execute code off the queues



```
DispatchQueue.main.async {  
    completionHandler(pokemonData)  
}
```


Grand Central Dispatch (GCD)



- Our Responsibility
 - Create a Queue
 - Put blocks of code on a queue

APIs on top of the threading, as Queues

When developers use GCD to submit tasks, the system automatically determines whether to assign the task to an E-core or a P-core based on the nature of the task and the current workload.

For instance, a computation-intensive task might be assigned to a P-core, while a low-priority background task might go to an E-core.

Different Queues

- Two different types:
 - **Main Queue**
 - The most important one of all. The queue where all blocks of code that affect the UI must be run on.
 - **Background Queues**
 - Used to queue up any long-lived, non UI tasks.
 - Often running simultaneously and in parallel with the main UI queue

– Main Queue

- `DispatchQueue.main`

– Background Queues

- `DispatchQueue.global(qos:QoS)`

- QoS: Quality of services

- » **.userInteractive:** Highest priority, use this class for tasks or queues that interact with the user
- » **.userInitiated:** Second only to user-interactive tasks. Assign this class to tasks that provide immediate results for somethings the user is doing

Creating Queues

- **Background Queues**

- `DispatchQueue.global(qos:QoS)`

- QoS: Quality of services

- » **.userInteractive:** Highest priority, use this class for tasks or queues that interact with the user
- » **.userInitiated:** Second only to user-interactive tasks. Assign this class to tasks that provide immediate results for somethings the user is doing
- » **.utility:** Have a lower priority. Assign this qos class to tasks that do not prevent the user from continuing to user your app
- » **.background:** Have the lowest priority of all tasks. Use when to perform work whilte your app is running the background (e.g., maintenance tasks or cleanup)

Background Queues

- .userInteractive
- .userInitiated
- **.default**
- .utility
- .background

Adding code to Queue

- Synchronous

- When a work item is executed synchronously with the sync method, the program waits until execution finishes before the method call returns.

```
DispatchQueue.main.sync{  
    //Code to be executed  
}
```

- Asynchronous

- When a work item is executed asynchronously with the async method, the method call returns immediately.

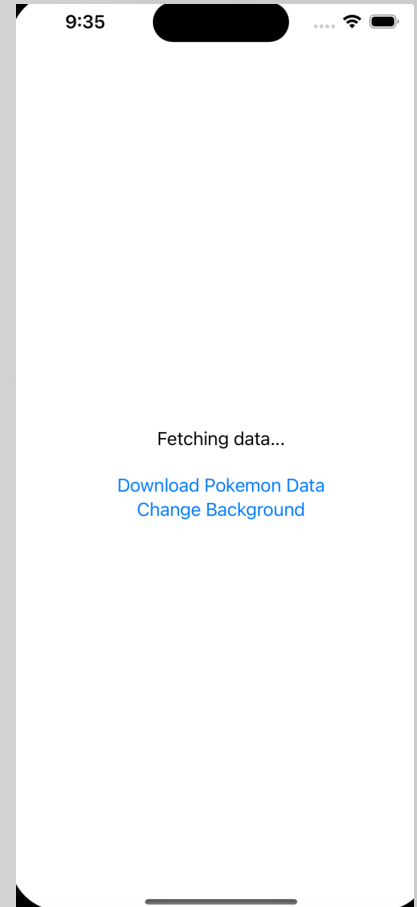
```
DispatchQueue.global().async{  
    //Code to be executed  
}
```

Example (ConcurrencyDemo)

```
func DownloadPokemonData() -> Void{
    DispatchQueue.global(qos: .background).async{
        sleep(2)

        let fetchedData = "Data fetched successfully!"

        // Update the UI on the main thread
        DispatchQueue.main.async {
            data = fetchedData
        }
    }
}
```



Drawback of GCD

- With GCD, the code can involve multiple nested closures, especially when chaining multiple asynchronous operations. This can lead to a "callback hell" situation, making the code harder to read and maintain.

```
import Dispatch

func fetchData(completion: @escaping (String?) -> Void) {
    DispatchQueue.global().asyncAfter(deadline: .now() + 1) {
        completion("User Data")
    }
}

func fetchUserPosts(userData: String, completion: @escaping ([String]?) -> Void) {
    DispatchQueue.global().asyncAfter(deadline: .now() + 1) {
        completion(["Post 1", "Post 2"])
    }
}

func fetchCommentsForPost(post: String, completion: @escaping ([String]?) -> Void) {
    DispatchQueue.global().asyncAfter(deadline: .now() + 1) {
        completion(["Comment 1 for \(post)", "Comment 2 for \(post)"])
    }
}
```

Implement the following:

1. Fetch user data.
2. Once we have the user data, we fetch their posts.
3. For each post, we then fetch its comments.

Callback hell

```
// Usage
fetchUserData { userData in
    if let userData = userData {
        fetchUserPosts(userData: userData) { userPosts in
            if let userPosts = userPosts {
                fetchCommentsForPost(post: userPosts[0]) { comments in
                    if let comments = comments {
                        print("Fetched Comments for \(userPosts[0]): \(comments)")
                    }
                }
                fetchCommentsForPost(post: userPosts[1]) { comments in
                    if let comments = comments {
                        print("Fetched Comments for \(userPosts[1]): \(comments)")
                    }
                }
            }
        }
    }
}
```

Implement the following:

1. Fetch user data.
2. Once we have the user data, we fetch their posts.
3. For each post, we then fetch its comments.

Callback hell

```
// Usage
fetchUserData { userData in
    if let userData = userData {
        fetchUserPosts(userData: userData) { userPosts in
            if let userPosts = userPosts {
                fetchCommentsForPost(post: userPosts[0]) { comments in
                    if let comments = comments {
                        print("Fetched Comments for \(userPosts[0]): \(comments)")
                    }
                }
                fetchCommentsForPost(post: userPosts[1]) { comments in
                    if let comments = comments {
                        print("Fetched Comments for \(userPosts[1]): \(comments)")
                    }
                }
            }
        }
    }
}
```

The nesting becomes deep and intertwined, making the code harder to read. This is a simple example, but in real-world scenarios, the nesting can become much deeper, leading to the "**callback hell**" or "**pyramid of doom**" situation.

Asynchronous Programming with async/await

- async/await introduced in Swift 5.5
- In Swift, the async/await feature enables you to write cleaner asynchronous code. It's like saying, "Hey, you go do this task, but let me know when you're finished so I can use the result."

```
func fetchData() async -> Data {  
    let data: Data = await loadData()  
    return data  
}  
  
Task {  
    let data = await fetchData()  
    // use the data  
}
```

In the above example, await tells Swift that fetchData is a time-consuming operation, and it should 'await' until it's finished before continuing.


```
import Dispatch
```

```
// Async functions
```

```
func fetchData() async -> String? {  
    await Task.sleep(1_000_000_000) // Simulating async delay  
    return "User Data"  
}
```

```
func fetchUserPosts(userData: String) async -> [String]? {  
    await Task.sleep(1_000_000_000) // Simulating async delay  
    return ["Post 1", "Post 2"]  
}
```

```
func fetchCommentsForPost(post: String) async -> [String]? {  
    await Task.sleep(1_000_000_000) // Simulating async delay  
    return ["Comment 1 for \(post)", "Comment 2 for \(post)"]  
}
```

```
// Usage in an async context
```

```
@MainActor // Ensuring this runs on the main thread (useful in the context of SwiftUI or UIKit)
```

```
func fetchAllDataAndPrint() async {  
    if let userData = await fetchData() {  
        if let userPosts = await fetchUserPosts(userData: userData) {  
            for post in userPosts {  
                if let comments = await fetchCommentsForPost(post: post) {  
                    print("Fetched Comments for \(post): \(comments)")  
                }  
            }  
        }  
    }  
}
```

```
// To call the function, you would typically do it from another async context.
```

```
// For instance, in SwiftUI's `Task` modifier or in an `async` function in your app logic.
```

```
var body: some View {  
    List(comments, id: \.self) { comment in  
        Text(comment)  
    }  
    .task {  
        await fetchAllDataAndUpdateUI()  
    }  
    .navigationTitle("User Comments")  
}
```

Better Error Handling

- **GCD:** Typically, error handling requires closures and traditional error callback mechanisms.
- **async/await:** You can use try, catch directly with asynchronous functions, making error handling more intuitive.

```
// Function that coordinates fetching all data
func fetchAllDataAndUpdateUI() async {
    do {
        if let userData = try await fetchUserData() {
            if let userPosts = try await fetchUserPosts(userData: userData) {
                for post in userPosts {
                    if let comments = try await fetchCommentsForPost(post: post) {
                        print("Fetched Comments for \(post): \(comments)")
                    }
                }
            }
        }
    } catch {
        print("An error occurred: \(error.localizedDescription)")
    }
}
```

without async/await

```
// Function that coordinates fetching all data using callbacks
func fetchAllDataUsingCallbacks() {
    fetchUserData { userData, error in
        if let error = error {
            print("Error fetching user data: \(error.localizedDescription)")
            return
        }
        if let userData = userData {
            fetchUserPosts(userData: userData) { userPosts, error in
                if let error = error {
                    print("Error fetching user posts: \(error.localizedDescription)")
                    return
                }
                if let userPosts = userPosts {
                    for post in userPosts {
                        fetchCommentsForPost(post: post) { comments, error in
                            if let error = error {
                                print("Error fetching comments for \(post): \(error.localizedDescription)")
                                return
                            }
                            if let comments = comments {
                                print("Fetched Comments for \(post): \(comments)")
                            }
                        }
                    }
                }
            }
        }
    }
}
```

Asynchronous Programming with async/await

- The use of async/await in Swift allows for more concise and readable code when performing asynchronous operations. Below is a demonstration of how you can incorporate async/await into our FetchAPI project:

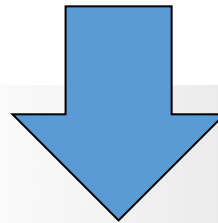
```
private func fetchAPIData(for pokemonID: Int) async throws -> PokemonData {  
    let url = URL(string: "https://pokemon.wcpc.fun/id/\(pokemonID)")!  
  
    let (data, _) = try await URLSession.shared.data(from: url)  
    return try JSONDecoder().decode(PokemonData.self, from: data)  
}
```

```
private func fetchAPIData(completionHandler: @escaping (PokemonData) -> Void, pokemonID:
Int) {
    let url = URL(string: "https://pokemon.wcpc.fun/id/\(pokemonID)")!

    URLSession.shared.dataTask(with: url) { (data, response, error) in
        guard let data = data else { return }

        do {
            let pokemonData = try JSONDecoder().decode(PokemonData.self, from: data)

            // Move to the main thread
            DispatchQueue.main.async {
                completionHandler(pokemonData)
            }
        } catch {
            print(error.localizedDescription)
        }
    }.resume()
}
```

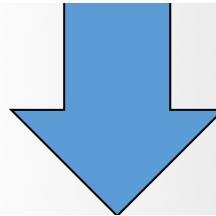


```
private func fetchAPIData(for pokemonID: Int) async throws -> PokemonData {
    let url = URL(string: "https://pokemon.wcpc.fun/id/\(pokemonID)")!

    let (data, _) = try await URLSession.shared.data(from: url)
    return try JSONDecoder().decode(PokemonData.self, from: data)
}
```

```
// Initializer fetches data for a Pokémon based on the given Pokémon ID.
init(PokemonID: Int) {
    fetchAPIData(completionHandler: { PokemonData in
        self.updateState(with: PokemonData)
    }, pokemonID: PokemonID)
}

func fetchNew(PokemonID: Int) {
    fetchAPIData(completionHandler: { PokemonData in
        self.updateState(with: PokemonData)
    }, pokemonID: PokemonID)
}
```



```
// Initializer fetches data for a Pokémon based on the given Pokémon ID.
init(PokemonID: Int) {
    Task {
        do {
            let PokemonData = try await self.fetchAPIData(for: PokemonID)
            self.updateState(with: PokemonData)
        } catch {
            print(error.localizedDescription)
        }
    }
}

func fetchNew(PokemonID: Int) {
    Task {
        do {
            let PokemonData = try await self.fetchAPIData(for: PokemonID)
            self.updateState(with: PokemonData)
        } catch {
            print(error.localizedDescription)
        }
    }
}
```



```
// Initializer fetches data for a Pokémon based on the given Pokémon ID.
init(PokemonID: Int) {
    fetchAPIData(completionHandler: { PokemonData in
        self.updateState(with: PokemonData)
    }, pokemonID: PokemonID)
}
```

Note that with these changes, the network call is now performed using `async/await`, which makes the code more straightforward and eliminates the need for completion handlers.

The use of `Task { ... }` ensures the asynchronous operation runs on a background thread

```
// Initializer fetches data for a Pokémon based on the given Pokémon ID.
init(PokemonID: Int) {
    Task {
        do {
            let PokemonData = try await self.fetchAPIData(for: PokemonID)
            self.updateState(with: PokemonData)
        } catch {
            print(error.localizedDescription)
        }
    }
}

func fetchNew(PokemonID: Int) {
    Task {
        do {
            let PokemonData = try await self.fetchAPIData(for: PokemonID)
            self.updateState(with: PokemonData)
        } catch {
            print(error.localizedDescription)
        }
    }
}
```

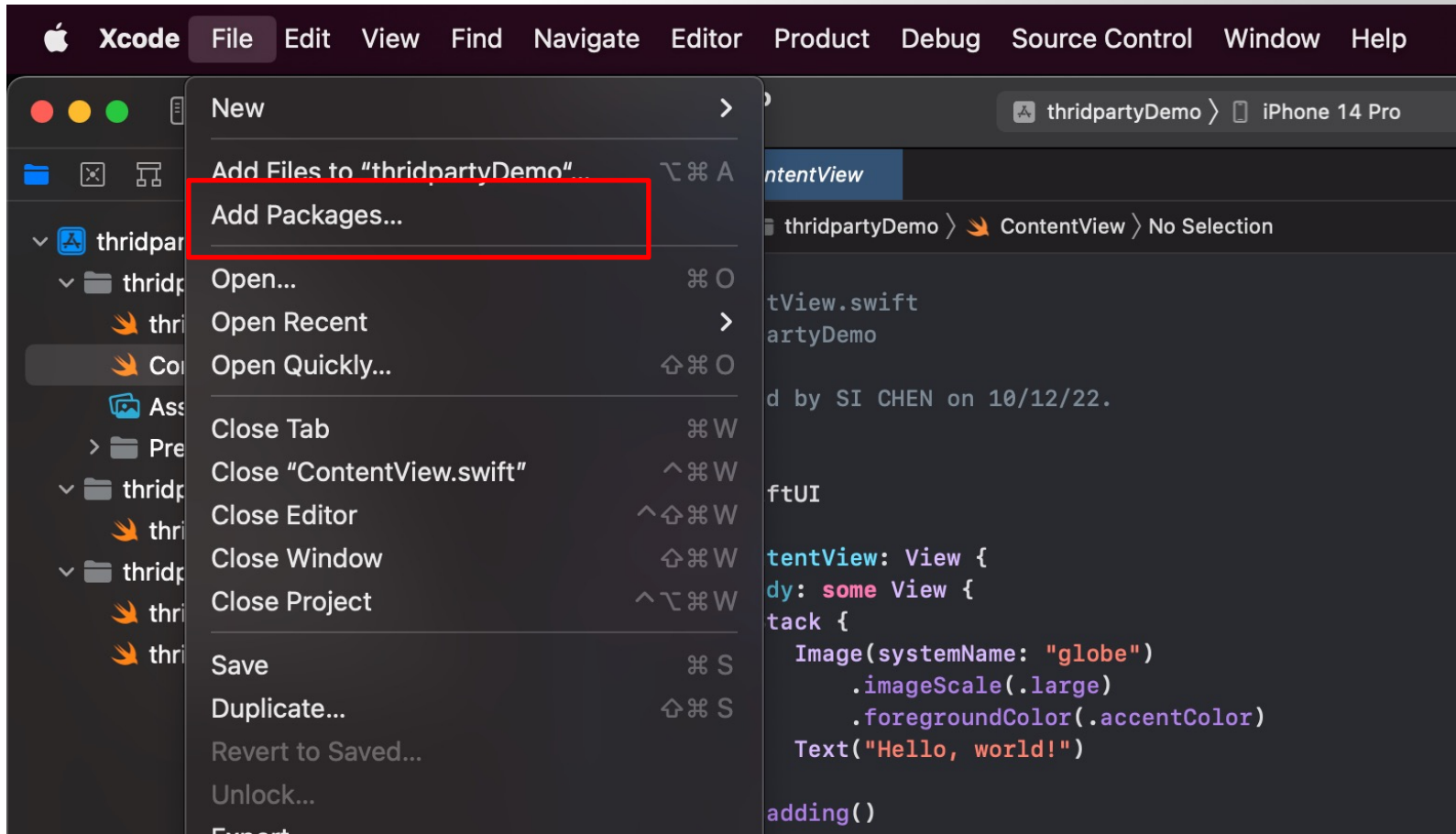
- Alamofire is a popular Swift-based HTTP networking library that simplifies the process of making network requests.

Install Alamofire:

Using Swift Package Manager (SPM):

1. Open your project in Xcode.
2. Navigate to File > Swift Packages > Add Package Dependency.
3. Click “github” and enter the Alamofire repository URL:
<https://github.com/Alamofire/Alamofire.git>
4. Follow the on-screen instructions to complete the installation.

Add Third-party packages



Add Third-party packages

The screenshot shows the 'Add Third-party packages' window in Xcode. The left sidebar lists 'Recently Used' (2) and 'Collections' (9), with 'Apple Swift Packages' selected. The main area shows a list of packages from Apple Swift Packages (developer.apple.com). The 'swift-algorithms' package is selected, and its details are shown on the right.

Apple Swift Packages
developer.apple.com

Search or Enter Package URL

swift-algorithms
Commonly used sequence and collection algorithms for Swift

swift-argument-parser
Straightforward, type-safe argument parsing for Swift

swift-atomics
Low-level atomic operations for Swift

swift-collections
Commonly used data structures for Swift

swift-crypto
Implements most of Apple's CryptoKit API for use across multiple platforms.

swift-nio
Supports development of asynchronous event-driven network applications that maintain high performance...

swift-numeric
Advanced mathematical types and functions for Swift

swift-system

swift-algorithms
Swift ☆ 4.61k

Version	Authors	Release Date	License	Repository
1.0.0	natecook1...	Sep 8, 2021	Apache v2.0	github.com

Dependency Rule: Up to Next Major Version (1.0.0 < 2.0.0)

Add to Project: thridpartyDemo

Description Release History Some metadata provided by [github.com](#)

Swift Algorithms

Swift Algorithms is an open-source package of sequence and collection algorithms, along with their related types.

Read more about the package, and the intent behind it, in the [announcement on swift.org](#).

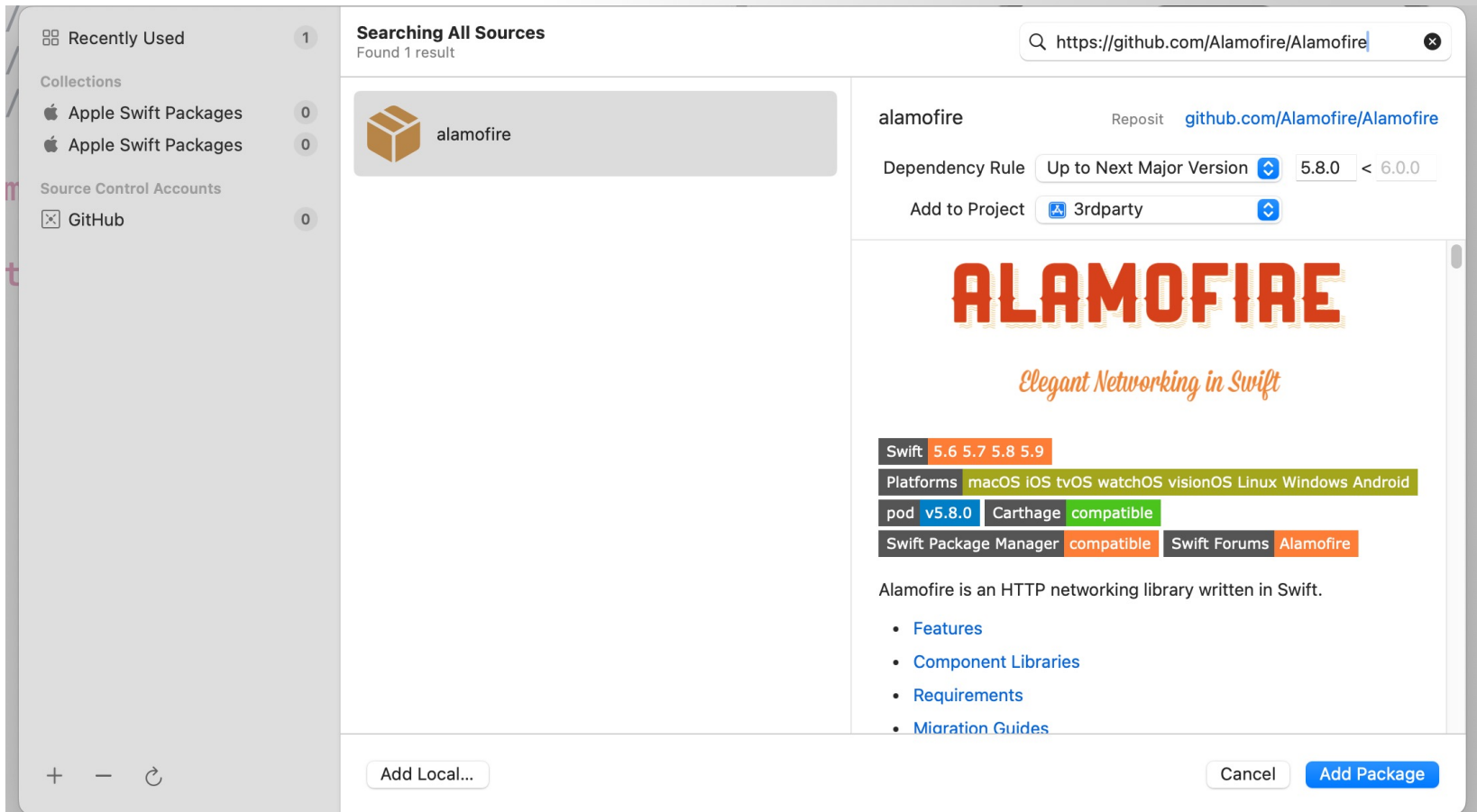
Contents

Combinations / permutations

- `combinations(ofCount:)` : Combinations of particular sizes of the elements in a collection.
- `permutations(ofCount:)` : Permutations of a particular size of

+ - ↺

Add Local... Cancel Add Package



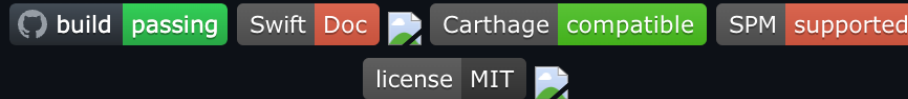
PokemonAPI

```
import PokemonAPI

struct ContentView: View {
    @State var pokemonName = ""
    @State var pokemonWeight = 0
    var body: some View {

        Text(pokemonName)
        Text(String(pokemonWeight))
        Button("Click"){
            PokemonAPI().pokemonService.fetchPokemon("150") { result in
                switch result {
                case .success(let pokemon):
                    self.pokemonName = pokemon.name ?? "" // bulbasaur
                    self.pokemonWeight = pokemon.weight ?? 0 //
                    bulbasaur
                case .failure(let error):
                    print(error.localizedDescription)
                }
            }
        }
    }
}

struct ContentView_Previews: PreviewProvider {
    static var previews: some View {
        ContentView()
    }
}
```

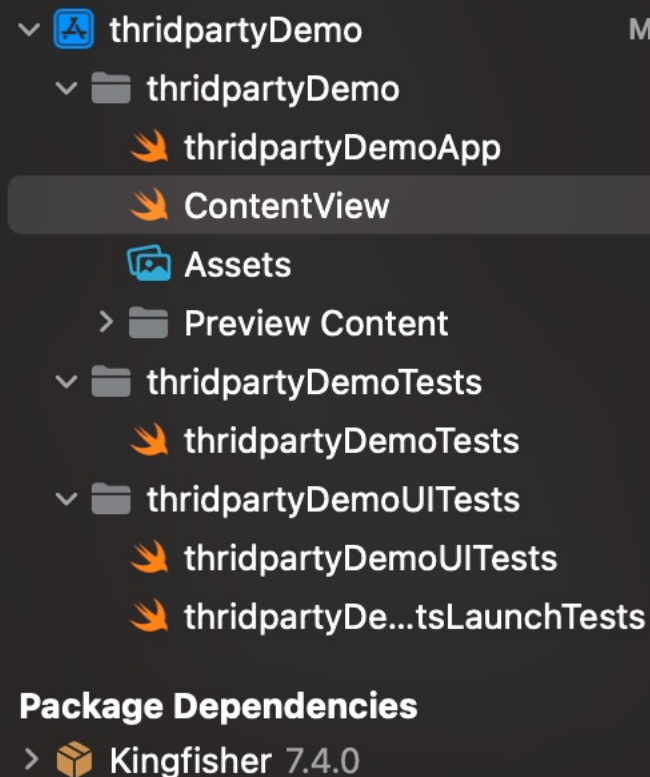


Kingfisher is a powerful, pure-Swift library for downloading and caching images from the web. It provides you a chance to use a pure-Swift way to work with remote images in your next app.

<https://github.com/onevcats/Kingfisher>

Swift Package Manager

- File > Swift Packages > Add Package Dependency
- Add `https://github.com/onevc/Kingfisher.git`
- Select "Up to Next Major" with "7.0.0"



Kingfisher

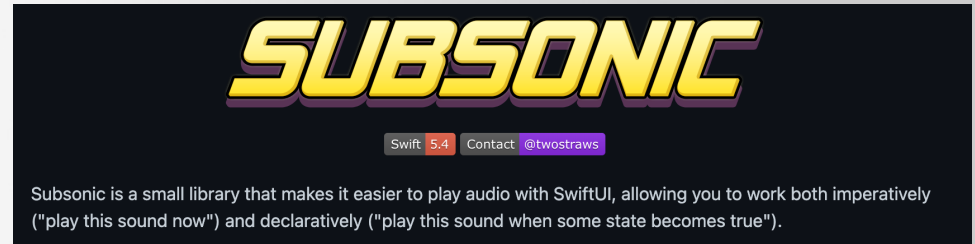
```
import SwiftUI
import Kingfisher

struct ContentView: View {
    var body: some View {
        VStack {
            Image(systemName: "globe")
                .imageScale(.large)
                .foregroundColor(.accentColor)
            Text("Hello, world!")
        }
        .padding()

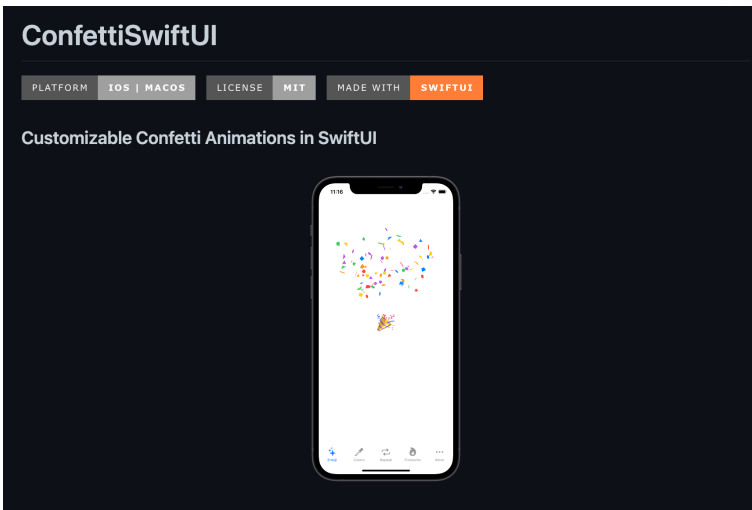
        KFIImage(URL(string:
            "https://www.wcupa
            .edu/_resources/includes/brand/images/headerLogo1.png"))!)
    }
}

struct ContentView_Previews: PreviewProvider {
    static var previews: some View {
        ContentView()
    }
}
```

Task: Try these 3rd party libraries



<https://github.com/twostraws/Subsonic>



<https://github.com/kinkofer/PokemonAPI>

Q & A

